

업데이트 된 게임 분석



우선 먼저 게임을 실행시킨 다음에 보니 이전과 다르게 총알이 날아가는 게 보였다. 또한 총알을 벽에 쏘니 벽에 총알이 닿으니 총알이 사라지는 것과 벽의 위치가 기존과 같다는 것 또한 확인했다.

```
C:\Users\정명훈\OneDrive\Desktop\게임 분석 프로젝트\dist2\cheat_ref.exe
# 4 ( 81, 527) hp100 <ME>
- ?꿀개 뽕뽕뽕 (?6? ???ㄴ빛) tick=15012 ---
# 2 ( 81, 527) hp 50
# 3 ( 81, 437) hp100
# 4 ( 81, 527) hp100 <ME>
- ?꿀개 뽕뽕뽕 (?6? ???ㄴ빛) tick=15013 ---
# 2 ( 81, 527) hp 50
# 3 ( 81, 437) hp100
# 4 ( 81, 527) hp100 <ME>
- ?꿀개 뽕뽕뽕 (?6? ???ㄴ빛) tick=15014 ---
# 2 ( 81, 527) hp 50
# 3 ( 81, 437) hp100
# 4 ( 81, 527) hp100 <ME>
- ?꿀개 뽕뽕뽕 (?6? ???ㄴ빛) tick=15015 ---
# 2 ( 81, 527) hp 50
```

그 다음에는 cheat_ref.exe파일을 실행시켰더니 게임 화면에 ghost가 뜨면서 스크린에 유령이 나타나는 데 이 때 유저 좌표와 체력을 출력하는 데 한 번 총으로 쏘도 체력이 변하지 않는 걸 봐서 처음 체력값만 가지고 있는 듯 했다. 또한 이후에 새 유저를 만들어 로그 인해도 로그인이 되지 않고 유령이 아닌 유저의 좌표 정보가 계속 바뀌는 등 신기한 일이 일어났다.

그 다음에는 client.exe파일을 한 번 정적 분석을하기로 결정했다. 가장 먼저 import부분을 보니 import부분이 방대한 걸 보아서 따로 패킹같은 작업을 한 거 같지는 않아서 바로 send함수로 xref로 갔더니

```
486     while ( v41 > v43 );
487     *(_WORD *)&Buffer[6] = v44;
488     send(g_sock, Buffer, v41, 0);
489     Thread = CreateThread(nullptr, 0, rx_thread, nullptr, 0, nullptr);
490     if ( Thread )
491         CloseHandle(Thread);
492     v47 = fopen_0("client.cfg", "w");
493     v48 = v47;
494     if ( v47 )
495     {
        fprintf(v47, "%s\n%s\n", stream, v286);
    }
```

이전과 같이 뭔가 정보를 send함수를 통해서 보내고 각 로그인 객체별로 스레드를 만들어서 관리하는 건 기존의 코드와 같다는 생각이 들었다. 그 다음에 아래 부분의 코드를 보니 너무 친절하게 변수 이름이 나와 있어 여기가 서버에게 전송할 것들을 정의하는 곳이라는 생각이 들었다.

```
503     while ( 1 )
504     {
505         v51 = 1;
506         while ( (unsigned int)SDL_PollEvent(v290) )
507             v51 = (v290[0] != 256) & (unsigned __int8)v51;
508         v266 = 0;
509         v268 = v51;
510         MouseState_0 = SDL_GetMouseState_0(&v275, &v276);
511         KeyboardState_0 = (_BYTE *)SDL_GetKeyboardState_0(0);
512         EnterCriticalSection(&g_lock);
513         v53 = _data_start__;
514         v265 = g_wall_count;
515         Size = 8LL * g_wall_count;
516         v253 = g_pl_count;
517         pszAddrString = g_bul_count;
518         if ( g_pl_count <= 0 )
519             break;
520         v54 = (const __m128i *)&g_pl;
521         v55 = 0;
```

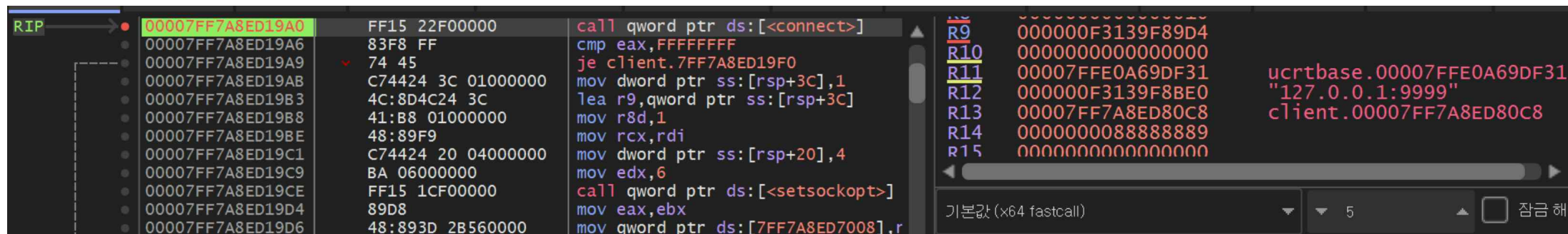
```

548 LABEL_94:
549     memcpy_0(v299, &g_walls, Size);
550     LeaveCriticalSection(&g_lock);
551     if ( v56 != -1 )
552     {
553         v66 = 0;
554         v65 = 0;
555         v158 = *(_DWORD *)&v300[24 * v56 + 4];
556         v254 = &v300[24 * v56];
557         v159 = *(_DWORD *)&v300[24 * v56 + 8];
558         *(double *)v66.m128_u64 = (double)v158;
559         *(double *)v65.m128_u64 = (double)v159;
560         v68 = atan2_0((double)(v276 - v159), (double)(int)(v275 - v158));
561         v67 = (int)(180.0 * v68 / 3.141592653589793); // 각도를 재는 부분
562         goto LABEL_96;

```

이 부분을 총알 각도를 정하는 부분이라는 생각이 들었다.

그다음에 나는 우선 이 프로그램에 파이썬 파일을 통해 연결하고 싶었고 연결 방식부터 파악하기 위해서 IDA의 IMPORT부분이 그대로 남아있어서 CONNECT쪽으로 가서 그 위치를 복사해 X64DBG에 중단점을 걸고 실행해 보았다.



그랬더니 이런 화면이 떴고 여기서 나는 connect 구조체 정보가 connect(

SOCKET s, // RCX

const struct sockaddr *name, // RDX

int namelen // R8

);와 같은 구조라는 정보를 ai한테 얻어서 rdx를 dump에서 따라가기를 눌러서 앞의 16바이트 정보를 확인해 보니

주소	Hex	ASCII
000000F3139F89D0	02 00 27 0F 7F 00 00 01	..'......
000000F3139F89E0	30 66 AD D1 FD 01 00 00	0f.Ný.....
000000F3139F89F0	07 01 00 00 00 00 00 00	ç-Ñý..
000000F3139F8A00	89 88 88 88 00 00 00 00	(í÷..
000000F3139F8A10	01 00 00 00 00 00 00 00	ÈúÑý..
000000F3139F8A20	07 01 00 00 00 00 00 00	ç-Ñý..
000000F3139F8A30	01 00 00 00 00 00 00 00	÷..

위와 같이 02 00 27 0F 7F 00 00 01이 나오고 이를 2바이트씩 분리해서 해석을 해보면 x86/64는 리틀 엔디안 기법을 사용하기 때문에 맨 처음 2바이트는 02 00 -> 0x0002 이고 Windows Socket에서 Address Family 값 2는 AF_INET이고 이는 바로 IPv4를 사용한다는 의미야. 그래서 여기서 sockaddr 구조체가 IPv4용 소켓 주소 구조체 sockaddr_in라는 걸 알 수 있고 그 구조는

```
struct sockaddr_in {
    short          sin_family;    // 2 byte

    unsigned short sin_port;      // 2 byte

    struct in_addr  sin_addr;     // 4 byte

    char           sin_zero[8];   // 8 byte
};
```

R8	0000000000000000000010
R9	0000000F3139F89D4
R10	0000000000000000000000

위와 같이 생겼기 때문에 크기를 다 더하면 0x10 = 이전 위의 connect 구조체의 namelen = 0x10과 동일하다. 따라서 더욱더 이 구조체가 sockaddr_in라는 걸 확신할 수 있으며 이 구조체 정보를 이용해서 나머지 바이트들을 해석해 보면 27 0F 는 포트 즉 네트워크 순서대로 해석해야 하기 때문에 빅 엔디안으로 해석하면 0x270F = 9999이고 7F 00 00 01또한 ip주소로 빅 엔디안으로 해석하면 127.0.0.1이 나온다. 이렇게 분석 시에는 첫 번째 두 바이트가 ipv4를 가리키는지 ipv6을 나타내는지 가장 중요하다. 왜냐하면 그 뒤의 바이트 해석이 완전히 달라지기 때문이다.

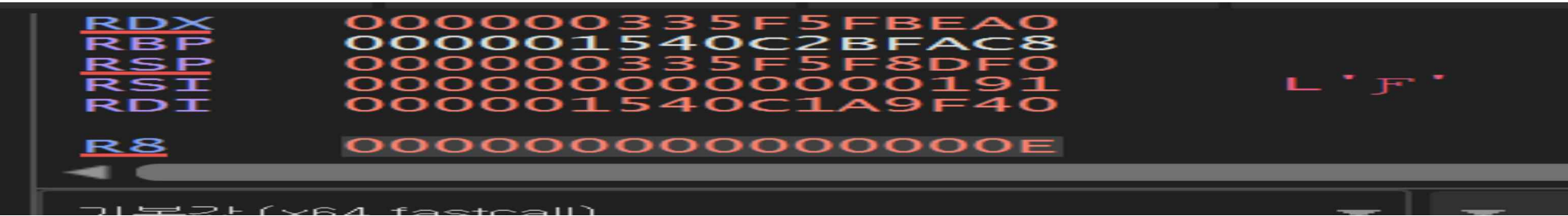
여기서 내가 안 중요한 사실은 api호출시 함수 prototype → 인자 순서 → x64 호출 규약 → 레지스터 값 순서대로 보는 것이 핵심이라는 거다. 어떻게 보면 너무 당연한 소리일지 모르겠지만 이렇게 분석을 해야 레지스터들의 의미가 더 명확히 들어온다.

그 다음에는 이제 어떤 데이터를 보내고 있는지 확인하기 위해서 SEND함수로 이동해서 레지스터의 값을 확인해 보니 int send(SOCKET s, // RCX

const char *buf, // RDX ← 실제 전송 데이터

int len, // R8 ← 데이터 길이

int flags // R9
);는 다음과 같은 프로토 타입을 가지고 이 정보를 이용해 RDX값을 덤프에서 따라가기를 해 분석해 보니



주소	Hex	ASCII
000000335F5FBEA0	47 47 02 01 06 00 13 2C 6D 65 32 72 77 65 00 00	GG.....,me2rwe..
000000335F5FBEB0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
000000335F5FBEC0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
000000335F5FBED0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	

길이인 14바이트 내용을 추출하니 000000335F5FBEA0 -> 47 47 02 01 06 00 13 2C 6D 65 32 72 77 65 00 00 GG.....,me2rwe..로 47 47 | 02 | 01 | 06 00 | 13 2C | 6D 65 32 72 77 65 이렇게 대충 나누어 볼 수 있는데 06 00은 이름의 길이 같아서 이걸 다른 이름 meme로 로그인을 해 보니

.text:00007FF7A8ED2A2A client.exe:\$2A2A #202A

주소	Hex	ASCII
000000D1CB3FBBA0	47 47 02 01 04 00 78 FB 6D 65 6D 65 00 00 00 00	GG...xûmeme...
000000D1CB3FBBB0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
000000D1CB3FBBC0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
000000D1CB3FBBD0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
000000D1CB3FBBE0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	

r8 = 12고 중간 2비트가 04 00임을 통해 5-6번 비트가 이름의 길이 정보라는 걸 확인했다. 그리고 02 01도 매번 반복이 되니 상관이 없는데 문제는 바로 5-6번째 비트가 이름이 변할 때마다 값이 변하는 데 이게 어떤 값인지를 완전히 파악하지 못했다. 이를 파악하기 위해서 xdbg와 ida를 찾아본 결과

```

485     J
486     while ( v42 > v44 );
487     *&Buffer[6] = v45;
488     send(g_sock, Buffer, v42, 0);
489     Thread = CreateThread(nullptr, 0, rx_thread, n

```

RAX	00000000FFFFFFB78	"meme"
RBX	000000C972DF8E50	
RCX	000000C972DFBD9C	
RDY	0000000000000000	

이 부분에서 rax의 하위 2비트 fb78을 리틀 엔디안 방식으로 저장했다는 사실을 알게 된 후 v45값을 추적해 보니

```

477     v44 = 0;
478     v45 = '\xFF\xFF\xC0\xDE';
479     do
480     {
481         v46 = *v43;
482         ++v44;
483         ++v43;
484         v45 = __ROL2__(v46 + v45, 3) ^ 0x5A5A;
485     }
486     while ( v42 > v44 );
487     *&Buffer[6] = v45;
488     send(g_sock, Buffer, v42, 0);

```

이렇게 버퍼의 값을 가지고 무언가 일종의 코드를 간단하게 shift + xor조합으로 만드는 거 같다는 느낌이 들었다. 그래서 이걸 keygen으로 파이썬 코드로 한 번 짜보기로 했다. 우선 xdbg로 이 do while loop으로 돌려본 결과 buffer의 값을 한바이트씩 읽어서 xor해서 저장한다는 걸 알 수 있었다. 이 정보를 활용해 코드를 짜니

```
def rol16(value, n):
```

```
    return ((value << n) | (value >> (16 - n))) & 0xFFFF
```

```
text = input("문자열 입력: ")
```

```
length = len(text) + 8
```

```
data = b'Wx47Wx47Wx02Wx01' + len(text).to_bytes(2, 'little') + b'Wx00Wx00' + text.encode()
```

```
key = int.from_bytes(b'WxC0WxDE', byteorder='big')
```

```
for byte in data:
```

```
    #print(byte.to_bytes())
```

```
    key = rol16(byte + key, 3) ^ 0x5A5A
```

```
print(key.to_bytes(2,'little').hex())
```

문자열 입력 : meme

78fb

문자열 입력 : me2rwe

132c

다음과 같은 코드를 짜고 돌리니 key값이 제대로 나오는 걸 확인할 수 있었다. 이를 통해서 서버에 보내는 정보에 대해서 완벽히 파악이 됐다.

그 다음에 서버에 파이썬 파일을 이용해서 한 번 연결해 보기 위해 dist2_client.py로 keygen코드를 이용해서 서버에 패킷을 보내면

```
sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.connect(('localhost', 9999))
#이름 설정
nickname = b'Bot'
#이름에 따른 길이 설정 리틀 엔디안으로 설정
payload = b'GG\x02\x01' + len(nickname).to_bytes(2, 'little') + keygen(nickname) + nickname
sock.settimeout(5.0)
```

```
b'GG\x02\x08\x01\x00r;\x00'
b'GG\x02\x05\x0f\x00\x18\x08`-\x00\x00\x01\x04Q\x00\x0f\x02d\x00\x00\x00\x01'
b'GG\x02\x08\x01\x00r;\x00'
b'GG\x02\x05\x0f\x00\x18(a-\x00\x00\x01\x04Q\x00\x0f\x02d\x00\x00\x00\x01GG\x02\x08\x01\x00r;\x00'
b'GG\x02\x05\x0f\x00c\xc8b-\x00\x00\x01\x04Q\x00\x0f\x02d\x00\x00\x00\x01'
b'GG\x02\x08\x01\x00r;\x00'
b'GG\x02\x05\x0f\x00c\xe8c-\x00\x00\x01\x04Q\x00\x0f\x02d\x00\x00\x00\x01GG\x02\x08\x01\x00r;\x00'
b'GG\x02\x05\x0f\x00F\x88d-\x00\x00\x01\x04Q\x00\x0f\x02d\x00\x00\x00\x01'
b'GG\x02\x08\x01\x00r;\x00'
```

```
[server] listening on :9999
[server] player 1 (Bot) joined
[server] player 1 left
[server] player 2 (Bot) joined
[server] player 2 left
[server] player 3 (Bot) joined
[server] player 4 (Bot) joined
[server] player 3 left
```

위의 화면처럼 잘 접속되는 걸 확인할 수 있다. 이로서 1단계 목표인 파이썬 파일로 exe파일을 실행하지 않고도 서버에 접속하기는 성공을 했다.

그 다음 나의 2번째 목표는 이전에 했던 것처럼 나머지 유저들의 위치와 벽의 위치를 알아내서 pygame tool을 이용해 화면에 표시하

는 거다. 이 목표를 달성하기 위해서는 먼저 pygame을 실행시키고 그 화면에서 단축키를 눌러가며 방향키 별 값들을 먼저 찾아야 한다.

```
569     v255 = nullptr;
570 LABEL_96:
571     LOBYTE(v9) = KeyboardState_0[26] != 0;
572     if ( KeyboardState_0[22] )
573         LODWORD(v9) = v9 | 2;
574     if ( KeyboardState_0[4] )
575         LODWORD(v9) = v9 | 4;
576     if ( KeyboardState_0[7] )
577         LODWORD(v9) = v9 | 8;
578     Ticks = SDL_GetTicks();
579     v71 = Ticks;
580     if ( v54 >= 0 && (Ticks - v270) > 0xF )
581     {
582         Buffer[8] = v9;
583         v228 = v51;
584         v229 = v51;
585         *&Buffer[9] = v68;
586         *Buffer = '\x03\x03\x02GG';
587         v230 = '\xFF\xFF\xC0\xDE';
588         do
589         {
590             v231 = *v229++;
591             v230 = __ROL2__(v231 + v230, 3) ^ 0x5A5A;
592         }
593         while ( v229 != &Buffer[11] );
594         *&Buffer[6] = v230;
595         (send)(g_sock, v51, 11, 0);
596         if ( (MouseState_0 & 1) != 0 && (v71 - last_fire_ms_0) > 0x12B )
597         {
598             last_fire_ms_0 = v71;
599             v232 = -16162;
600             *Buffer = 0x204024747LL;
601             *&Buffer[8] = v68;
```

이 부분에서 방향키와 관련된 정보를 암호화된 KEYGEN과 함께 같이 보낸다는 가설을 세우고 우선 SEND에서 어떤 걸 보내는 지 확인하기 위해 XDBG를 실행하고 SEND함수 부분에 중단점을 걸고 레지스터 값을 이전의 SEND의 프로토타입을 통해 확인해 보니

RDX

00000051BEDFBEA0

RBP

000001BF1AD1FAC8

RSP

00000051BEDF8DF0

RSI

00000000000001B1C

RDI

00000051BEDFBEA0

R8

0000000000000000B

덤프 1

덤프 2

덤프 3

덤프 4

덤프 5

주시 1

로컬

주소	Hex	ASCII
00000051BEDFBEA0	47 47 02 03 03 00 CF 82 00 EC FF 00 00 00 7E 40	GG...ï..ÿ...~@
00000051BEDFBEB0	00 00 00 00 00 40 85 40 00 00 00 00 00 00 7E 40	@.@.....~@
00000051BEDFBEC0	00 00 00 00 00 40 85 40 00 00 00 00 00 40 85 40	@.@.....~@

이름이 mememe일 때는 rdx레지스터의 11바이트를 확인하니 이렇게 나왔고 중단점을 풀고 다른 방향키를 눌렀더니

.text:00007FF7A8ED4379 C:\rent.exe.\$4379 #3B79

덤프 1

덤프 2

덤프 3

덤프 4

덤프 5

주시 1

로컬

주소

Hex

ASCII

00000051BEDFBEA0

47 47 02 03 03 00 C6 6C 00 74 00 FF 00 00 00 00

GG....Æl.t.ÿ....

00000051BEDFBEB0

00 00 00 00 68 16 15 43 00 00 FA 43 D2 D2 D7 FF

....h..c..úCòðxÿ

584

v229 = v51;

585

*&Buffer[9] = v68;

586

*Buffer = '\x03\x03\x02GG';

587

v230 = '\xFF\xFF\xC0\xDE';

588

do

589

{

그리고 여기서 상위 6비트는 위의 스크린샷의 /xG/xG/x02/x03/x03과 같이 방향키와 상관없이 이제 나머지 변하는 부분 중

```

.MODEL_70.
LOBYTE(v9) = KeyboardState_0[26] != 0;
if ( KeyboardState_0[22] )
    LODWORD(v9) = v9 | 2;
if ( KeyboardState_0[4] )
    LODWORD(v9) = v9 | 4;
if ( KeyboardState_0[7] )
    LODWORD(v9) = v9 | 8;
Ticks = SDL_GetTicks();
v71 = Ticks;
if ( v54 >= 0 && (Ticks - v270) > 0xF )
{
    Buffer[8] = v9;
    ...
}

```

이 코드를 봤을 때 9번째 값이 keybord정보를 받는 걸 보아 방향키를 받는 정보라는 걸 확인하고 각 방향키를 누르며 변화를 관찰해 보는 게 쉽지가 않아서 조금 더 정적분석을 해보니 기존의 buffer배열에서

r	488	19	send(g_sock, Buffer, v42, 0);
r	500	12	v51 = Buffer;
w	582	10	Buffer[8] = v9;
w	585	12	* &Buffer[9] = v68;
w	586	11	*Buffer = 'Wx03Wx03Wx02GG';
o	593	27	while (v229 != &Buffer[11]);
w	594	12	* &Buffer[6] = v230;
w	600	13	*Buffer = 0x204024747LL;
...	604	14	* &Buffer[10] = v60;

처음 연결한 정보에서 앞의 5바이트는 47 47 02 03 03으로 바꾸고 7-8번째 바이트는 v230즉 이전과 유사하게 기존 버퍼 정보를 담은 v51과 xor + rotate 한 값을 집어 넣고 있었고 11-12번째 바이트에는 각도 정보가 들어가는 것처럼 보였고

Addr	Line	Column	Pseudocode line
w	561	10	v68 = (180.0 * v69 / 3.141592653589793); // 각도 부분
w	567	15	LOWORD(v68) = 0;
r	585	24	* &Buffer[9] = v68;
r	601	26	* &Buffer[8] = v68;

9번째 바이트에는 방향키 정보가 들어간다는 사실까지 확인을 했다. 그리고 6번째 바이트는 기존 값을 쓰는데 기존 값이 00이라는 것 또한 알아냈다. 이제 이러한 정보를 이용해서 파이썬 코드로 작성하기 전에 우선 각도 정보를 어떻게 얻는지부터 파악을 할 필요가 있다.

그러기 위해서 각도 정보를 얻기 위해서 마우스 위치 정보와 현재 위치 정보를 얻어야 되기 때문에 recv함수를 분석하기로 결정하고 recv함수를 보니

```

41 {
42     v3 = recv(g_sock, &Src[v2], 0x2000 - v1, 0);
43     if ( v3 <= 0 )
44         return 0;
45     v4 = v1 + v3;
46     v1 = v4;
47     if ( v4 <= 7 )
48     {
49         v18 = &Src[v5 + 13];
50         v19 = &g_pl;
51         v20 = (__int64)&v10[10 * (v17 - 1) + 15];
52         do
53         {
54             v21 = *v18;
55             v22 = *(__int16 *) (v18 + 1);
56             v18 += 10;
57             v19 += 6;
58             v23 = *(unsigned __int16 *) (v19 - 2);
59         } while ( v21 < 0 );
60     }
61 }

```

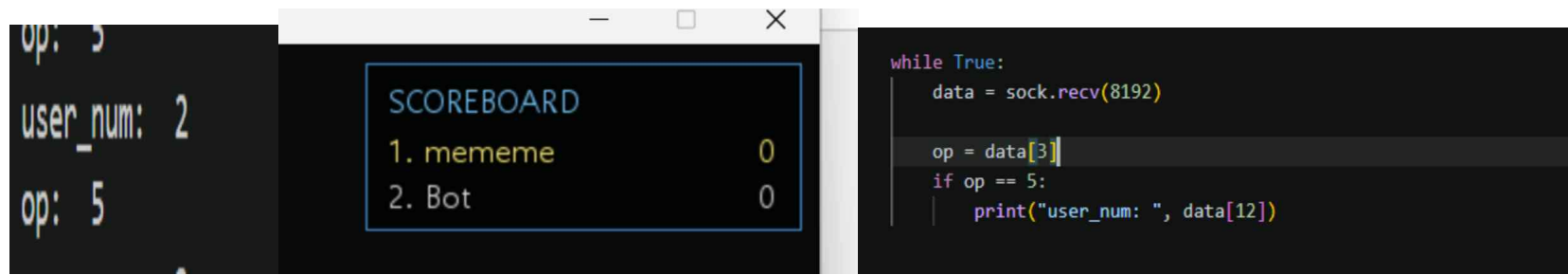
이렇게 g_pl관련된 정보를 받고 있는 게 보였다.

```
10         while ( v4 < 7 )
11         {
12             v6 = &Src[v5];
13             v7 = *(_QWORD *)v6;
14             if ( (unsigned __int16)*(_QWORD *)v6 == 'GG' )
15                 break;
16             ++v5;
17             v1 = v4 - v5;
18             if ( v4 - v5 <= 7 )
19                 goto LABEL_13;
20         }
21         v8 = WORD2(v7);
22         if ( WORD2(v7) + 7 >= v1 )
23             goto LABEL_14;
24         v9 = v7 >> 24;
25         v10 = &Src[v5 + 8];
26         if ( (_BYTE)v9 == 2 )
27         {
28             v11 = Src[v5 + 12];
29             00001320 rx thread:62 (7FF7A8ED1D20)
30         }
```

이 코드를 자세히 분석해본 결과 receive packet의 첫 두 바이트가 GG임을 확인한 다음에 shift연산 후 가장 하위 비트를 cmp하는 방식으로 각각 어떤 정보를 switch문으로 보내는 걸 확인했고 이 때 하위 1비트가 receive packet의 4번째 바이트 receive[3]이라는 것 또한 xdbg를 활용해 레지스터를 보고 파악을 했다.

```
94         1
95         switch ( (_BYTE)v9 )
96         {
97             case 5:
98                 v15 = Src[v5 + 12];
99                 EnterCriticalSection(&g_lock);
100                 v16 = 32;
101                 if ( (unsigned __int8)v15 <= 0x20u )
102                     v16 = v15;
103                 v17 = v16;
104                 g_pl_count = v16;
105                 if ( v15 )
106                 {
107                     v18 = &Src[v5 + 13];
108                     v19 = &g_pl;
109                     v20 = (__int64)&v10[10 * (v17 - 1) + 15];
110                     do
111                     {
112                         v21 = *v18;
```

그 중에서 내가 알고 싶었던 값은 `receive[3] == 5`일때 값이었기 때문에 이전 코드에서 `while`문으로 `receive`를 계속해서 받으면서 `receive[3] == 5`일 때만 분석을 하기로 했다 이 때 `v15`는 32를 초과하는 경우 32개로 한정하는 걸 보아서 현재 로그인 중인 유저라고 추론을 했고 `xdbg`을 통해 확인해 본 결과



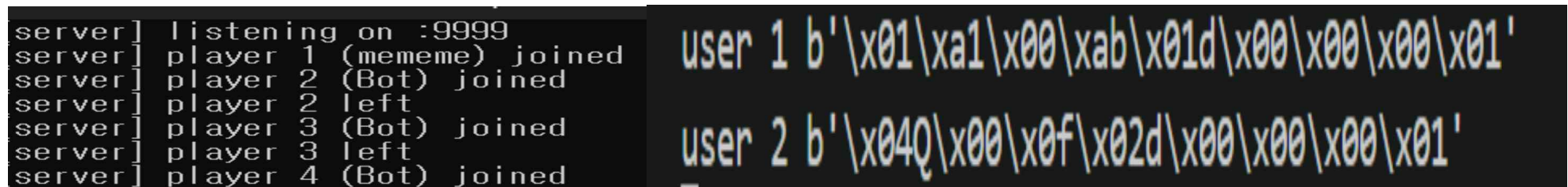
실제로 유저 수임이 확인이 됐다. 또한 `receive[13]`부터 유저 정보를 가져온다는 것도 알 수 있었다. 특이한 점은

```
if ( user_num )
{
    v18 = &Src[v5 + 13];
    v19 = &g_pl;
    v20 = (__int64)&v10[10 * (v17 - 1) + 15];
do
{
    v21 = *v18;
    v22 = *(__int16 *)(v18 + 1);
    v18 += 10;
    v19 += 6;
    v23 = *(unsigned __int16 *)(v18 - 3);
    *((_BYTE *)v19 - 24) = v21;
```

`v18`은 10바이트씩 정보를 넘기고 있는데 특이하게 `v19`는 타입이 `dword`라 `+6`은 24바이트씩 정보를 저장하고 있다는 점이다.

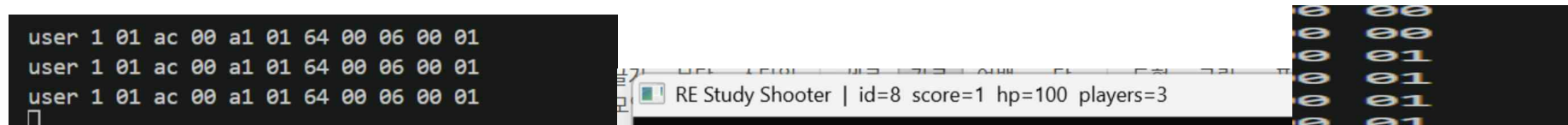


이 점을 이용해 `python`으로 값을 입력받아보니 값들을 하나씩 확인을 해보니



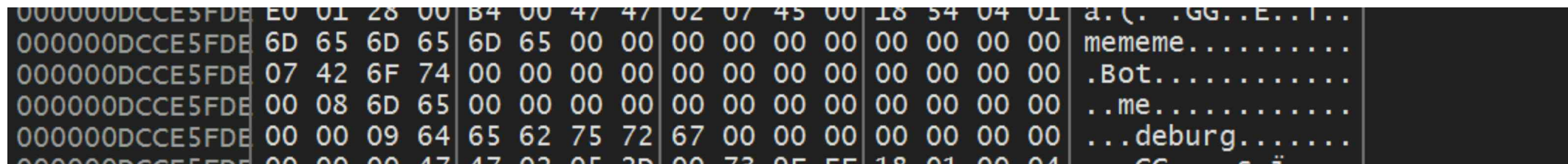
첫 번째 바이트는 유저의 아이디를 가리키는 걸 알아냈다.

그 다음엔 exe를 통해 접속한 유저로 마우스를 움직이면서 각도 정보를 바꿔보았는데 그럴 때 기준으로 receive한 packet이 전혀 변하지 않아 적어도 각도와 관한 정보는 받지 않는다는 걸 알아냈다. 또한 상하로 움직일 때 4-5번째 비트값이 바뀐다는 점을 통해 4-5번째 비트 정보가 현재 y좌표라는 것을 알아냈고 그 다음에는 좌우로 움직일 때는 2-3번째 비트값이 바뀐다는 점을 통해 2-3번째 비트 정보가 현재 x좌표라는 걸 알 수 있었다.



그 다음값들은 좀 찾기가 어려웠는데 64 00이 부분은 16진수 기준으로 100으로 아래의 명시된 체력같았고 다른 유저로 총을 쏘서 체력을 줄이니 4B 00으로 준 걸로 보아 체력임을 확인했고 그 다음에는 06 00이 부분은 상대를 맞출 때마다 1씩 증가하는 걸 보아서 상대방을 맞힌 횟수이고 마지막 01은 죽는 순간 00으로 바뀌었다가 다시 부활하니 01로 나타나는 걸 보아 생존 여부를 나타내는 변수라는 것을 확인했다.

이렇게 얻은 정보들 중 X,Y값과 MOUSE_X,MOUSE_Y위치를 이용해서 각도를 계산하려고 하니 또다른 문제가 생겼다 내가 내 자신의 이름을 통해서 내 id를 알아내야 내 자신의 위치를 구할 수 있다는 점이다. 이 문제를 해결하기 위해 recive부분에 중단점을 걸고 rdx 레지스터를 살피니



역시 receive packet으로 내 이름 정보도 보내고 있었고 여기서 ida를 통해 정적인 분석을 해보니

```

127     case 7:
128         v25 = (unsigned __int8)*v10;
129         EnterCriticalSection(&g_lock);
130         if ( (_BYTE)v25 )
131         {
132             v26 = &Src[v5 + 9];
133             do
134             {
135                 v27 = (unsigned __int8)*v26;
136                 v28 = _mm_loadu_si128((const __m128i*)(v26 + 1));
137                 v26 += 17;
138                 v27 *= 32;
139                 g_names[v27 + 16] = 0;
140                 *(__m128i*)&g_names[v27] = v28;
141             }
142             while ( &v10[17 * (v25 - 1) + 18] != v26 );

```

이 부분에서 이름 정보를 저장하는 작업을 하는 듯 했다. 17바이트씩 저장하는 듯해서 파이썬에서 동작을 시켜 값을 확인하니 실제로

```

pygame 2.6.1 (SDL 2.28.4, Python 3.13.0)
Hello from the pygame community. https://www.pygame.org/contribute.html
01 6d 65 6d 65 6d 65 00 00 00 00 00 00 00 00 00
15 42 6f 74 00 00 00 00 00 00 00 00 00 00 00
09 64 65 62 75 72 67 00 00 00 00 00 00 00 00

```

```

server] player 20 left
server] player 21 (Bot) joined

```

```

C:\Users\성명훈\OneDrive\Desktop\게임 분석 프로젝트\dist2\server.
[server] listening on :9999
[server] player 1 (mememe) joined
[server] player 2 (Bot) joined
[server] player 2 left
[server] player 3 (Bot) joined
[server] player 3 left
[server] player 4 (Bot) joined
[server] player 4 left
[server] player 5 (Bot) joined
[server] player 5 left
[server] player 6 (Bot) joined
[server] player 6 left
[server] player 7 (Bot) joined
[server] player 8 (me) joined
[server] player 9 (debug) joined

```

id와 이름을 처음 유저가 접속할 때마다 갱신해서 보내주는 걸 알 수 있었다. 이를 통해서 내 이름과 id와 이름을 매핑하면 드디어 각도를 계산할 수 있다.

```

while running:
    for e in pygame.event.get():
        ✨ ✨ if e.type == pygame.QUIT:
            running = False
    mouse_x, mouse_y = pygame.mouse.get_pos()

    data = sock.recv(8192)

    op = data[3]
    if op == 5:
        cur_x, cur_y = 1,1
        NUM = data[12]

    if op == 7:
        My_id = -1
        for i in range(NUM):
            if(nickname == data[17 * i +10: 17 * i + 13 ]):
                My_id = data[17*i + 9]

```

이렇게 코드를 짜고 실행하니 이상하게 실행이 안되고 op를 찍어봐도 잘 나오지 않았다. 그 이유는 간단하다. recv한 패킷이 while문일 때는 우연히 한 패킷씩 전달이 됐지만 pygame이벤트 처리를 하는 등의 일을 하니 여러 패킷이 동시에 전달되서 제대로 출력이 안되는 것이었다. 이를 해결하기 위해 header에서 payload길이를 나타내는 필드의 값을 이용해서 다시 코드를 짰다(이 부분은 ai의 도움을 일부 받았다.)

그 후에는 packet[3]값이 7일때 이름과 id 매핑 정보를 획득하고 그 이후 packet[3]값이 5이고 id매핑 정보가 확인이 된 경우에 pygame의 마우스 좌표를 가져오는 함수와 내 현재 좌표를 receive packet을 통해서 획득하고 atan함수와 식을 적용해 계산하는 코드를 작성해 결국 각도 정보를 얻어내는 데 성공했다.

```

86         if op == 5:
87             NUM = packet[12]
88             if(MY_ID != -1):
89                 mouse_x, mouse_y = pygame.mouse.get_pos()
90                 cur_x, cur_y = packet[10*(MY_ID + 1) + 4: 10*(MY_ID + 1) + 6], packet[10*(MY_ID + 1) + 6: 10*(MY_ID + 1) + 8]
91
92                 cur_y = int.from_bytes(cur_y, 'little', signed=True)
93                 cur_x = int.from_bytes(cur_x, 'little', signed=True)
94
95                 ins = math.atan2(mouse_y - cur_y, mouse_x - cur_x)
96                 angle = int(180.0 * ins / math.pi)
97                 angle_bytes = angle.to_bytes(2, byteorder='little', signed=True)
98
99
100         elif op == 7:
101             print(NUM)
102             for i in range(NUM):
103                 print(nickname, ":", packet[17 * (i) + 10 : 17 * (i) + 13])
104                 if(nickname.hex() == packet[17 * (i) + 10 : 17 * (i) + 13].hex()):
105                     MY_ID = packet[17 * i + 9]
106                     print(MY_ID)
107                     break

```

그 이후에는 이제 이전에 나왔던 이전과 비슷한 xor패턴을 적용하는 비트 부분을 봐야 하는데

```

//-----
v71 = Ticks;
if ( v54 >= 0 && (unsigned int)(Ticks - v270) > 0xF )
{
    Buffer[8] = (char)v9;
    v228 = v51;
    v229 = v51;
    *(_WORD *)&Buffer[9] = v68;
    *(_QWORD *)Buffer = '\x03\x03\x02GG';
    v230 = '\xFF\xFF\xC0\xDE';
    do
    {
        v231 = (unsigned __int8)*v229++;
        v230 = __ROL2__(v231 + v230, 3) ^ 0x5A5A;
    }
    while ( v229 != &Buffer[11] );
    *(_WORD *)&Buffer[6] = v230;
}

```

w	502	6	v51 = Buffer;
r	585	17	v228 = v51;

00007FF7A8ED2AA1	4D:89F5	mov r13,r14	R12	000000800E9F8FB0
00007FF7A8ED2AA4	C78424 C4000000 0000	mov dword ptr ss:[rsp+C4],0	R13	00007FF7A8ED80C8
00007FF7A8ED2AAF	48:898424 E0000000	mov qword ptr ss:[rsp+E0],rax	R14	000000800E9FBE90
00007FF7A8ED2AB7	48:8D8424 24010000	lea rax,qword ptr ss:[rsp+124]	R15	0000000000000000
00007FF7A8ED2ABF	F244:0F100D 80570000	movsd xmm9,qword ptr ds:[7FF7A8EC]		

주소	Hex	ASCII
000000800E9FBE	47 47 02 01 06 00 8A 26 64 65 62 75 72 67 00 00	GG.....&debug..
000000800E9FBE	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
000000800E9FBE	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
000000800E9FBE	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
000000800E9FBE	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	

이전에 서버 접속할 때와 패턴이 굉장히 유사해서 v229값만 잘 추적하면 되겠다 싶었고 따라서 v51값을 추적하니 기존 버퍼값이고 그 위치에 xdbg로 접속해 중단점을 걸고 r14레지스터를 확인하니 위처럼 기존의 버퍼값 즉 내가 서버에게 처음 연결을 위해 send한 값이 그대로 저장되어 있음을 확인했다.

이제 그 정보와 기존의 xor함수 기법을 적용해서 파이썬 코드를 작성하다 보니 생각보다 많은 어려움을 겪었다 우선 op == 5일때 유저 id가 23이면 현재 서버에 총 유저가 2명만 있어도 패킷에 총 23명의 유저정보가 담길 줄 착각해서 MY_ID를 index로 이용해 현재 x,y좌표를 구한 결과 완전히 잘못된 결과를 얻었고 ida에 나온 내용을 ai도움 없이 작성하려고 하다보니 많은 실수를 하는 등 시행착오를 거쳤다. 그렇게 시행착오를 거쳐 코드를 완성하고

```

71         v255 = nullptr;
72 LABEL_96:
73     LOBYTE(v9) = KeyboardState_0[26] != 0;
74     if ( KeyboardState_0[22] )
75         LODWORD(v9) = (unsigned int)v9 | 2;
76     if ( KeyboardState_0[4] )
77         LODWORD(v9) = (unsigned int)v9 | 4;
78     if ( KeyboardState_0[7] )
79         LODWORD(v9) = (unsigned int)v9 | 8;
80     Ticks = SDL_GetTicks();

```

```

cur_x: 81 cur_y: 597 mouse_x: 0 mouse_y: 0
cur_x: 81 cur_y: 602 mouse_x: 0 mouse_y: 0
cur_x: 81 cur_y: 607 mouse_x: 0 mouse_y: 0
cur_x: 81 cur_y: 612 mouse_x: 785 mouse_y: 103
cur_x: 81 cur_y: 617 mouse_x: 429 mouse_y: 154
cur_x: 81 cur_y: 622 mouse_x: 149 mouse_y: 199
cur_x: 81 cur_y: 627 mouse_x: 39 mouse_y: 213
cur_x: 81 cur_y: 632 mouse_x: 5 mouse_y: 213
cur_x: 81 cur_y: 637 mouse_x: 0 mouse_y: 213
cur_x: 81 cur_y: 642 mouse_x: 0 mouse_y: 213
cur_x: 81 cur_y: 647 mouse_x: 0 mouse_y: 213

```

이 부분의 ida코드를 이용해 이동키 부분에 1,2,4,8를 넣어보니 1을 넣었을 때는 위로 이동했고 2를 넣었을 때는 아래로 4를 넣었을 때

는 왼쪽으로 8을 넣으면 오른쪽으로 간다는 걸 확인했다(위의 스크린샷을 2를 넣었을 때 검증값이다) 그 정보를 이용해 pygame함수로 각 wasd키가 눌렸을 때 이동을 하게 만드는 데 성공했다.

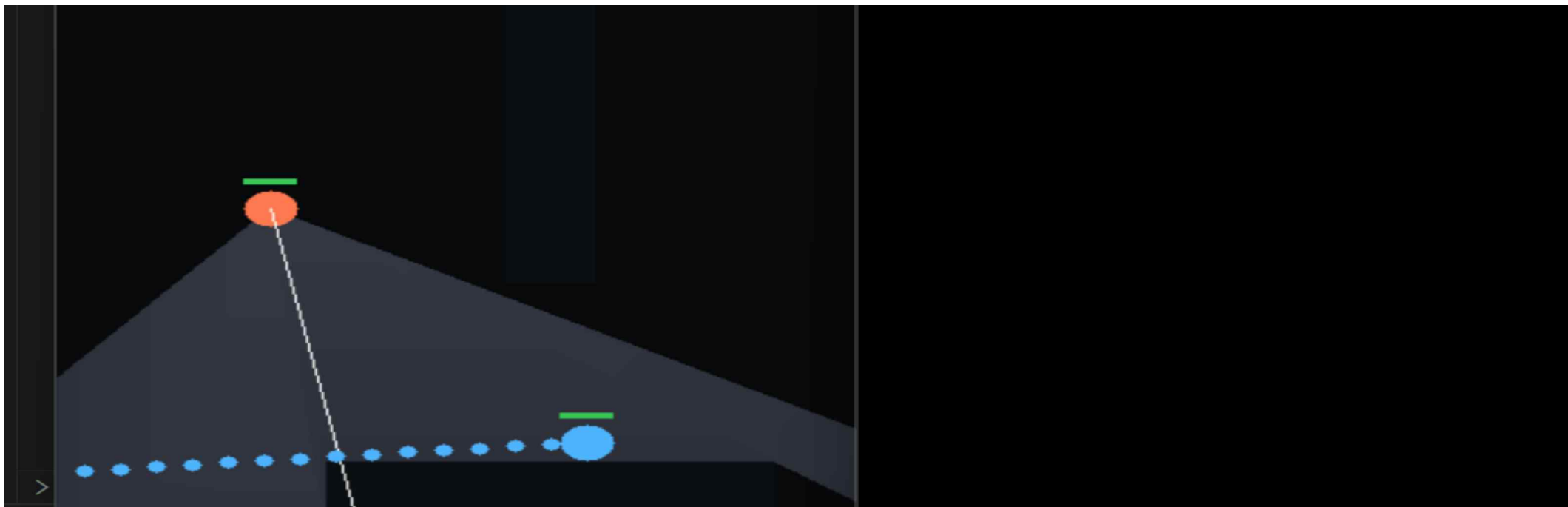
3단계로 이제는 단순히 이동하는 게 아니라 총을 쏘고 싶다. 따라서 다시 ida에 가서 총 쏘는 부분의 패킷을 보내는 형식을 자세히 분석해 보면

```
597 ((void (__fastcall *)(SOCKET, char *, __int64, _QWORD))send)(g_sock, v51, 11, 0);
598 if ( (MouseState_0 & 1) != 0 && (unsigned int)(v71 - last_fire_ms_0) > 0x12B )
599 {
600     last_fire_ms_0 = v71;
601     v232 = '\xFF\xFF\xC0\xDE';
602     *(_QWORD *)Buffer = '\x02\x04\x02GG';
603     *(_WORD *)&Buffer[8] = v68;
604     do
605     {
606         v233 = (unsigned __int8)*v228++;
607         v234 = v232 + v233;
608         LOWORD(v234) = __ROL2__(v234, 3) ^ 0x5A5A;
609         v232 = v234;
610     }
611     while ( v228 != v51 + 10 );
612     *(_WORD *)&Buffer[6] = v234;
613     ((void (__fastcall *)(SOCKET, char *, __int64, _QWORD))send)(g_sock, v51, 10, 0);
614 }
```

이전의 위치정보를 보낼 때와 굉장히 유사한 방식으로 우선 mouse_state == 1이고 마지막 총알 발사시간으로부터 0x12b 즉 299ms 후가 지났으면 즉 대략 0.3초가 지났으면 작동하게 되어 있고 그 안에 작동원리는 이전의 wasd 키보드 입력이 들어왔을 때와 거의 유사하고 약간 숫자만 바뀌었을 뿐이다. 또한 각도 정보를 이전 과정에서 구했기 때문에 쉽게 이 정보를 활용해 총알을 발사하는 코드를 작성할 수 있을 거 같다는 예감이 든다. 단지 현재 키 정보만 안 보내면 될 거 같다는 생각이 들었고 예상대로 코드를 작성하고

```
148 angle_bytes = angle.to_bytes(2, byteorder='little', signed=True)
149
150 payload_move_ins = bytearray(payload)
151 payload_move_ins[0:8] = b'GG\x02\x03\x03\x00\x00\x00'
152 payload_move_ins[9:11] = angle_bytes
153 payload_move_ins[8] = keyState
154
155 payload_shoot_ins = bytearray(10)
156 payload_shoot_ins[0:8] = b'GG\x02\x04\x02\x00\x00\x00'
157 payload_shoot_ins[8:10] = angle_bytes
158
159
160
161
162 payload_move = b'GG\x02\x03\x03\x00' + keygen_2(payload_move_ins) + keyState.to_bytes() + angle_bytes
163 sock.send(payload_move)
164
165 payload_shoot = b'GG\x02\x04\x02\x00' + keygen_2(payload_shoot_ins) + angle_bytes
```

실행하니 총알이 잘 나간다는 것 또한 확인할 수 있었다.



이제 4번째 단계로 총알이 나가는 것까지 패킷으로 작성을 했으니 그 다음으로는 이전에 각도를 계산하며 얻은 유저 좌표 정보를 이용해서 모든 유저의 위치 정보를 모두 PYGAME을 이용해서 표시해 보자



```

171
172     screen.fill((255, 255, 255))
173
174     for id, coord in players.items():
175         x, y = coord
176         print("id:", id)
177         color = (255, 0, 0) if id != MY_ID else (0, 255, 0)
178         pygame.draw.circle(screen, color, (x, y), 10)
179
180     if MY_ID in players:
181         print("yes")
182         pygame.draw.line(screen, (255,0,0), (mouse_x, mouse_y), (cur_x, cur_y), 5)
183
184     pygame.display.flip()

```

```

posloc = 0
mouse_x, mouse_y = pygame.mouse.get_pos()

for i in range(0, NUM ):
    ins_x, ins_y = packet[10*(i ) + 14: 10*(i ) + 16], packet[10*(i ) + 16: 10*(i ) + 18]
    ins_y = int.from_bytes(ins_y, 'little', signed=True)
    ins_x = int.from_bytes(ins_x, 'little', signed=True)

    player_id = int.from_bytes(packet[10*(i ) + 13: 10*(i ) + 14])
    players[player_id] = (ins_x, ins_y)

    if(packet[10 * i + 13] == MY_ID):
        posloc = i

```

기존에 작성한 코드를 약간 변형해서 작성했더니 생각보다 빠르게 구할 수 있었다.

이제 5단계로 벽을 표시하면 기본적인 핵을 만드는 것은 끝이 난다. 그러기 위해서는 우선 벽을 어디서 그리는지 언제 그리는지를 파악하기 위해서 ida를 이용해서 정적인 분석을 해야 한다.

우선 ida로 receive부분 근처를 보면 이렇게 벽의 개수와 벽의 정보를 메모리에 복사하는 게 보인다. 아마도 g_walls에 벽의 정보가 담긴 거 같다는 생각이 들었다. 또한 if문을 보니 op == 2일때 정보를 받는다는 것 또한 알 수 있었다. v13은 벽의 수를 나타내고 최대 64개가 최대 개수라는 것도 파악했다. 이 지점에서 우선 먼저 xdbg64를 사용해서 중단점을 걸고 값을 관찰해 보면 아래 디버거의 레지스터 값을 보면 우선 벽의 개수가 7개라는 걸 알 수 있다.

```

71 v10 = &Src[v5 + 8];
72 if ( (_BYTE)v9 == 2 )
73 {
74     v11 = Src[v5 + 13];
75     v12 = (unsigned __int8)*v10;
76     EnterCriticalSection(&g_lock);
77     v13 = v11;
78     if ( v11 > 0x40u )
79     {
80         v13 = 64;
81         v14 = 512;
82     }
83     else
84     {
85         v14 = 8LL * v11;
86     }
87     _data_start__ = v12;
88     g_wall_count = v13;
89     if ( (unsigned int)v14 >= 8 )
90         qmemcpy(&g_walls, &Src[v5 + 14], 8 * (v14 >> 3));
91     ((void (__fastcall *)(_RTL_CRITICAL_SECTION *))LeaveCriticalSection)(&g_lock);
92 }

```

007FF7A8ED1DDC	8905 FED10000	mov dword ptr ds:[7FF7A8EDEFE0],eax	RAX	0000000000000007
007FF7A8ED1DE2	83F9 08	cmp ecx,8	RBX	0000000000000000
007FF7A8ED1DE5	72 07	jb client.7FF7A8ED1DEE	RCX	0000000000000038
007FF7A8ED1DE7	48:C1E9 03	shr rcx,3		
007FF7A8ED1DEB	E348:A5	rep movsq		

그 다음에 이 부분을 보면 v11은 벽의 개수 7이고 g_walls에 이 벽의 개수 v11 = v14 >> 3만큼 *8바이트씩 총 56바이트를 복사하고 있음을 알 수 있고 이를 통해 벽 한개의 크기는 8바이트라는 걸 알 수 있다. 그렇다면 g_walls의 바이트가 어떻게 나누어져 해석되는지 확인하기 위해서 g_walls를 추적해 보면

```

551 memcpy_0(v300, &g_walls, Size);
552 LeaveCriticalSection(&g_lock);
553 if ( v57 != -1 )
554     r

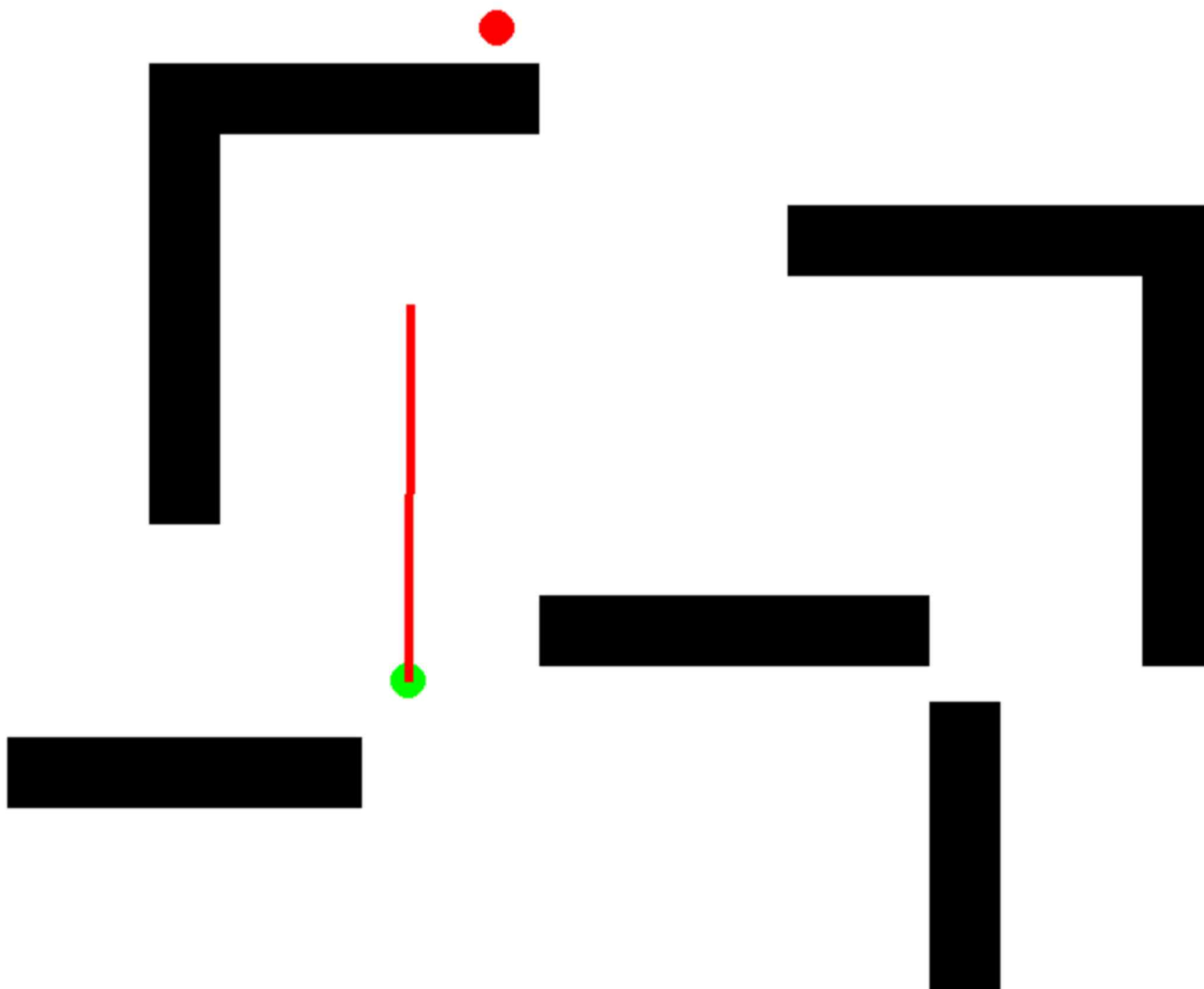
```

```
IDA View-A Pseudocode-A type library: mssdk64_win7 Strings Hex View-1 Local types Import
647 v77 = (unsigned __int16 *)v300;
648 v9 = (unsigned __int16 *)&v300[Size];
649 v78 = 49;
650 do
651 {
652     v79 = *v77;
653     v80 = 0;
654     v81 = v77[1];
655     v82 = 0;
656     v83 = v79 + v77[2];
657     *(__m128d *)Buffer = _mm_cvtepi32_pd(_mm_unpacklo_epi16(_mm_cvtsi32_si128(*(_DWORD *)v77), (__m128i)0LL));
658     v84 = 0;
659     v84.m128d_f64[0] = (double)v83;
660     v80.m128d_f64[0] = (double)v81;
661     v85 = v81 + v77[3];
662     v311 = _mm_unpacklo_pd(v84, v80);
663     v82.m128d_f64[0] = (double)v85;
664     v312 = _mm_unpacklo_pd(v84, v82);
665     v86 = 0;
666     v86.m128d_f64[0] = (double)v79;
667     v313 = _mm_unpacklo_pd(v86, v82);
668     if ( v78 <= 253 )
669     {
```

이걸 보면 정확히는 복잡하게 simd로 적혀있어 완벽한 의미파악은 어렵지만 대략적으로 시작 좌표를 하나 잡고 너비와 높이를 더해서 다른 점들을 구하는 걸 보니 시작 좌표가 왼쪽 끝점인 거 같다는 생각이 들어서 한번 파이썬 코드로 출력하고 검증해보기로 했다.

```
break
op == 2:
print(packet[13])
for i in range(0,7):
    print( int.from_bytes(packet[8*(i) + 14 : 8*(i) + 16], 'little'), " ", int.from_bytes(packet[8*(i) + 16 : 8*(i) + 18], 'little'), " ", int.from_bytes(packet[8*(i) + 18 : 8*(i) + 20], 'little'))
time.sleep(10)
```

(200 120 40 260), (200 120 220 40), (560 200 240 40), (760 200 40 260), (420 420 220 40),
(120 500 200 40), (640 480 40 180) 결과로 좌표와 너비처럼 보이는 값이 나왔고 이 값을 pygame으로 이동하면서 벽
을 부딪치며 확인해본 결과 실제로 그 위치에 벽이 있다는 걸 확인했다 이제 이 정보를 이용해서 벽을 pygame으로 그려보면



이렇게 기초적인 해를 만드는 데 성공했다. 이제 그 다음에 다음으로는 `cheat_ref.exe`파일을 한 번 분석해 보려고 실행을 시켰더니 그냥 내가 지금까지 해온 것들 정보들을 뛰우는 것 같아서 그걸 분석하는 것보다는 아까전에 체력관련 값을 무한히 늘려 불사신을 만들어보기로 했다.

5.내가 지금까지 한 건 기본적인 맵핵을 만드는 거였지만 이제는 추가적으로 모든 유저의 체력을 pygame화면에 표시하려고 한다. 이를 위해서는 이전에 RECEIVE PACKET을 분석하면서 얻은 체력정보를 우선 가져와야 하는데 나는 이미 앞에서 receive를 분석하며 체력이 어떤 바이트를 뜻하는 지 알기 때문에 이 정보를 이용해서 문제를 풀어보려고 한다. packet[3] == 5일때 6-7번이 체력 정보임으로 이를 이용해 ai의 도움을 약간 받아 체력바를 그리는 코드를 짜면 아래와 같은 스크린샷의 결과가 나오고

```
for i in range(0, NUM ):
    ins_x, ins_y, ins_hp = packet[10*(i ) + 14: 10*(i ) + 16], packet[10*(i ) + 16: 10*(i ) + 18], packet[10*(i ) + 18: 10*(i ) + 20]
    ins_y = int.from_bytes(ins_y, 'little', signed=True)
    ins_x = int.from_bytes(ins_x, 'little', signed=True)
    hp = int.from_bytes(ins_hp, 'little', signed=True)
    player_id = int.from_bytes(packet[10*(i ) + 13: 10*(i ) + 14])
    players[player_id] = (ins_x, ins_y, hp)

    if(packet[10 * i + 13] == MY_ID):
        posloc = i

cur_x, cur_y = packet[10*(posloc ) + 14: 10*(posloc ) + 16], packet[10*(posloc ) + 16: 10*(posloc ) + 18]
cur_y = int.from_bytes(cur_y, 'little', signed=True)
```

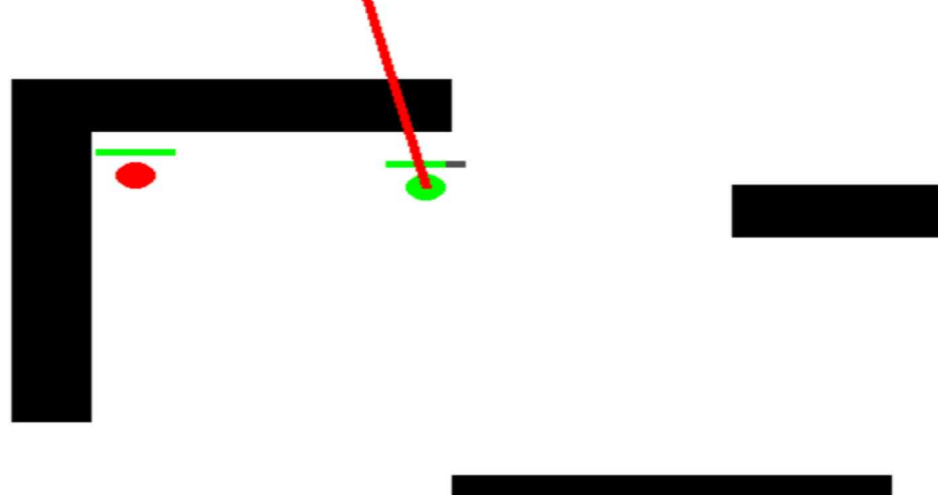
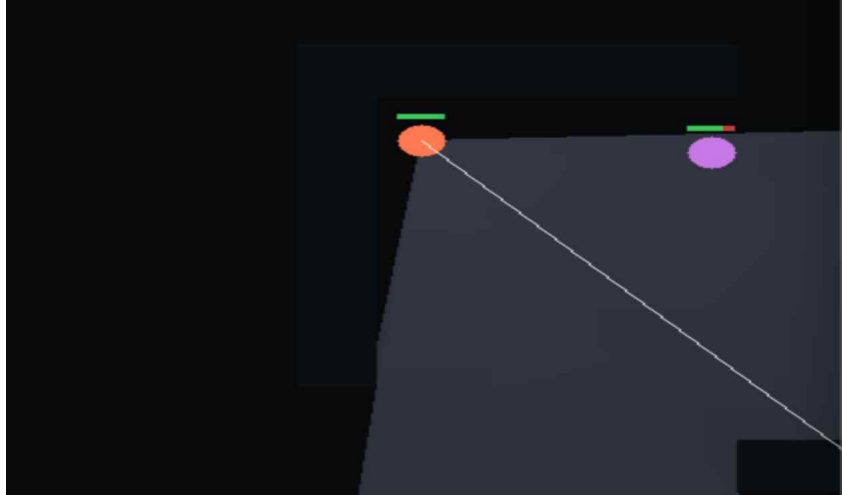
```
# 체력바 설정
bar_width = 40
bar_height = 5

bar_x = x - bar_width // 2
bar_y = y - 20

# 체력바 배경
pygame.draw.rect(
    screen,
    (80, 80, 80),
    (bar_x, bar_y, bar_width, bar_height)
)

# 현재 체력
pygame.draw.rect(
    screen,
    (0, 255, 0),
    (bar_x, bar_y, bar_width * hp / 100, bar_height)
)
```

그 다음에 화면을 출력하면



이렇게 뜬다. 결론적으로 이 정도면 완벽히 상대의 정보까지 파악하는 핵이 완성됐다. 이것이 변형된 게임에 대한 핵의 완성본이다.